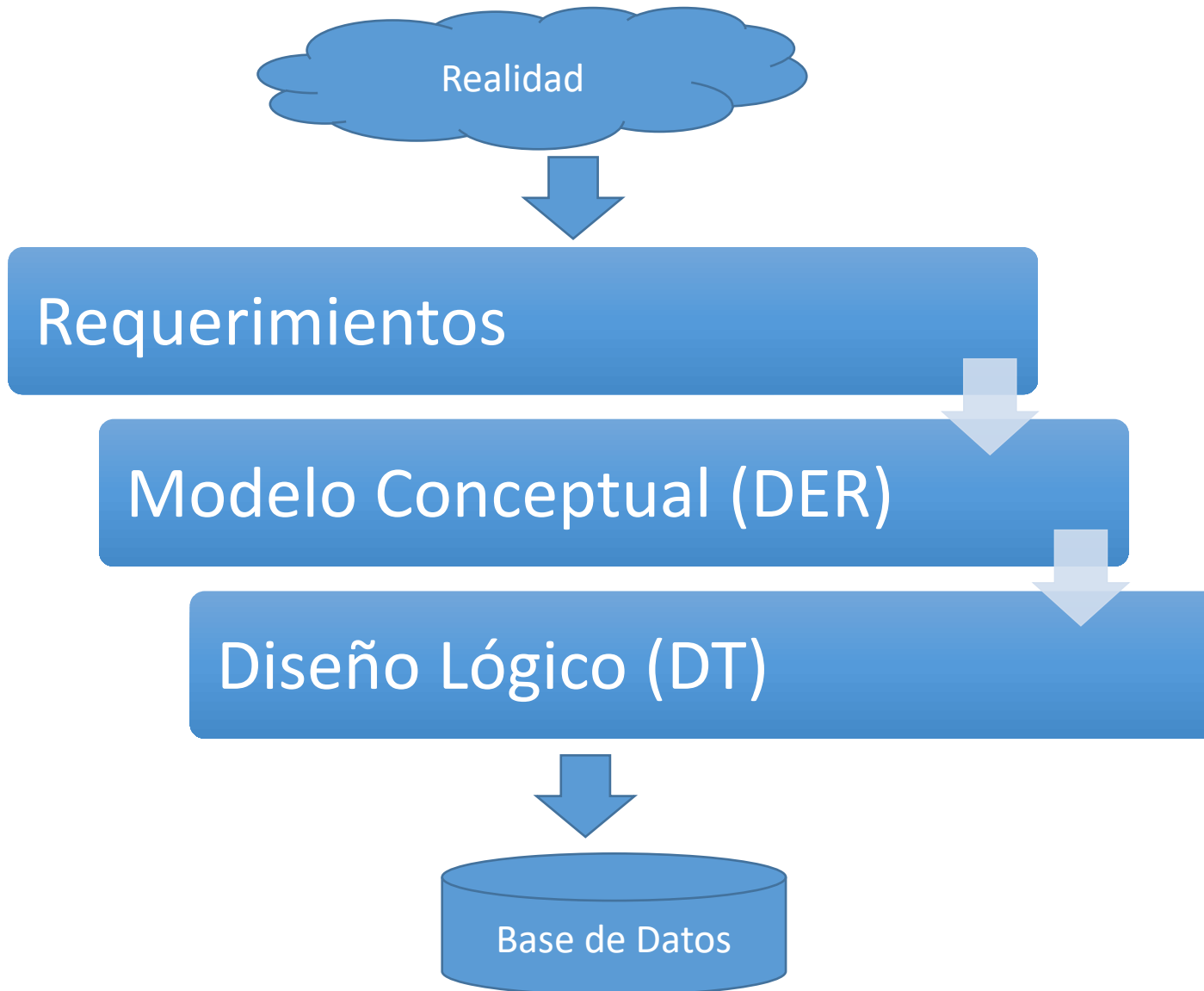


IMPLEMENTACIÓN DE UNA BASE DE DATOS

Lenguaje : DDL - DML

Implementación de una Base de Datos



Lenguaje de Implementación de BD

SQL (Structured Query Language): lenguaje de consulta estructurado que permite el acceso a un sistema de gestión de bases de datos relacional.

Sub-lenguajes SQL:

- ✓ **Data Definition Language (DDL)** comandos usados para definir/modificar la estructura de los objetos de la DB.

CREATE ALTER DROP

- ✓ **Data Manipulation Language (DML)** comandos para manipular los datos de la DB

INSERT UPDATE DELETE

- ✓ **Query Language (QL)** comando para realizar las consultas sobre los datos de la DB

SELECT

- ✓ **Data Control Language (DCL)** comandos para manejar permisos en la DB

GRANT REVOKE

- ✓ **Transaction Control Language (TCL)** comandos para manejar transacciones de la DB

COMMIT ROLLBACK SAVEPOINT

Los denominados dialectos de SQL están asociados a cada motor o RDBMS que se utiliza: Oracle, DB2, Informix, MySQL, Postgresql, entre otros...

Base de Datos

Objetos /Estructura DDL

Esquema

- Crear CREATE SCHEMA
- Borrar DROP SCHEMA
- Cambiar definición ALTER SCHEMA

Tablas

- Crear CREATE TABLE
- Borrar DROP TABLE..
- Cambiar nombre tabla ALTER TABLE.....
- Cambiar

Columnas

- Añadir ALTER TABLE ADD COLUMN.....
- Borrar ALTER TABLE DROP COLUMN.....
- Cambiar Nombre ALTER TABLE RENAME COLUMN TO.....
- Cambiar Tipo de dato ALTER TABLE ALTER COLUMN TYPE.....

Restricciones

- Añadir ALTER TABLE ALTER COLUMN..... SET NOT NULL
- ALTER TABLE..... ADD CONSTRAINT CHECK/FOREIGN/UNIQUE...
- Borrar ALTER TABLE ALTER COLUMN DROP NOT NULL
- ALTER TABLE DROP CONSTRAINTCHECK/FOREIGN/UNIQUE...

Datos DML

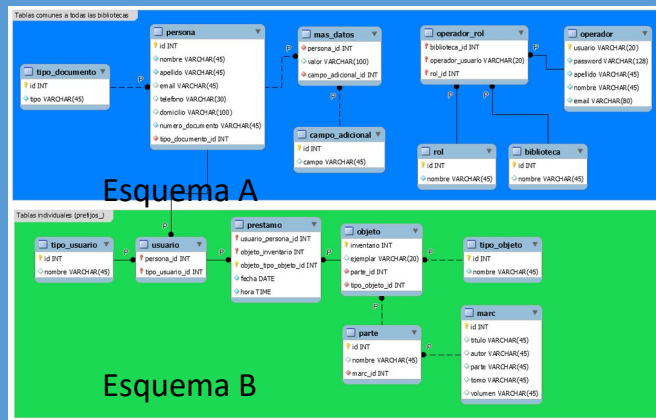
Filas

- Crear INSERT INTO VALUES (.....)
- Modificar UPDATE SET
- Borrar DELETE FROM

DDL- Esquema de una BD

El concepto de esquema SQL se incorporó por primera vez en SQL2 para agrupar las tablas y otras estructuras pertenecientes a la misma aplicación de base de datos. Un esquema SQL se identifica con un nombre de esquema.

Base de Datos
A-B



```
CREATE SCHEMA <nombre_esquema> [AUTHORIZATION  
<responsable_esquema>];
```

```
CREATE SCHEMA esquema A AUTHORIZATION Jperez;  
CREATE SCHEMA esquema B;
```

Cambiar la definición del esquema

```
ALTER SCHEMA <nombre_esquema> RENAME TO <nuevo_Nombre>  
ALTER SCHEMA <nombre_esquema> OWNER TO <nuevo_dueño>
```

```
ALTER SCHEMA esquema1 RENAME TO esquema_trabajo;
```

Borrar un esquema de una base de datos

```
DROP <nombre_esquema>;      Ej. DROP esquema 1;
```

Base de Datos

Objetos /Estructura DDL

Esquema

- Crear CREATE SCHEMA
- Borrar DROP SCHEMA
- Cambiar definición ALTER SCHEMA

Tablas

- Crear CREATE TABLE
- Borrar DROP TABLE..
- Cambiar nombre tabla ALTER TABLE.....
- Cambiar

Columnas

- Añadir ALTER TABLE ADD COLUMN.....
- Borrar ALTER TABLE DROP COLUMN.....
- Cambiar Nombre ALTER TABLE RENAME COLUMN TO.....
- Cambiar Tipo de dato ALTER TABLE ALTER COLUMN TYPE.....

Restricciones

- Añadir ALTER TABLE ALTER COLUMN..... SET NOT NULL
- ALTER TABLE..... ADD CONSTRAINT CHECK/FOREIGN/UNIQUE...
- Borrar ALTER TABLE ALTER COLUMN DROP NOT NULL
- ALTER TABLE DROP CONSTRAINTCHECK/FOREIGN/UNIQUE...

Datos DML

Filas

- Crear INSERT INTO VALUES (.....)
- Modificar UPDATE SET
- Borrar DELETE FROM

Crear una tabla

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio  
);
```

```
CREATE TABLE empleado (  
    DNI                integer,  
    fecha_Nac          date,  
    edad               integer ,  
    primer_nombre      varchar (20),  
    primer_apel        varchar (10),  
    segundo_apel       varchar(10)  
);
```



Restricción asociada con el dominio
de sus columnas o tributos

Los tipos de datos básicos disponibles para los
atributos son numérico, cadena de caracteres,
cadena de bits, booleano, fecha y hora

Empleado	
PK	<u>DNI</u> Fecha_Nac Edad Primer_Nombre Primer_Apel Segundo_Apel
FK	Id_Dpto

Crear una tabla: clave primaria (simple)

Restricciones:

Clave Primaria

Clave Foránea

Not Null

Clave Única

Check

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio CONSTRAINT <nombre_restricción> PRIMARY KEY,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio  
);
```

(a)

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio,  
    CONSTRAINT <nombre_restricción> PRIMARY KEY (atributo1_nombre)  
);
```

(b)

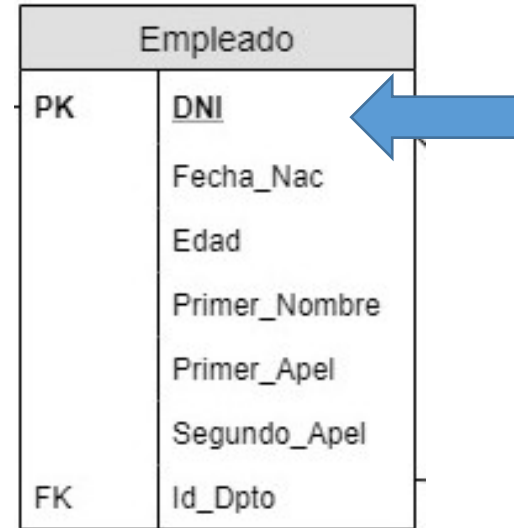
Restricciones:
Clave Primaria
Clave Foránea
Not Null
Clave Única
Check

Crear tabla: clave primaria (simple)-Ejemplo

```
CREATE TABLE empleado (  
    DNI integer CONSTRAINT pk_dni PRIMARY KEY, ← Nivel de columna  
    fecha_Nac date,  
    edad integer ,  
    primer_nombre varchar (20),  
    primer_apel varchar (10),  
    segundo_apel varchar(10));  
    (a)
```

```
CREATE TABLE empleado (  
    DNI integer,  
    fecha_Nac date,  
    edad integer ,  
    primer_nombre varchar (20),  
    primer_apel varchar (10),  
    segundo_apel varchar(10),  
    CONSTRAINT pk_dni PRIMARY KEY (DNI));  
    (b)
```

← Nivel de tabla



Clave Primaria (PK)
simple
Se define a nivel de
columna (a) o a nivel de
tabla (b)

Crear una tabla: clave primaria (compuesta)

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoPK1_nombre  tipo_dominio,  
    .....,  
    atributoPKN_nombre  tipo_dominio,  
    CONSTRAINT <nombre_restricción> PRIMARY KEY (atributoPK1_nombre,..., atributoPKN_nombre));
```

```
CREATE TABLE empleado (  
    tipo_Doc      varchar(3),  
    nro_doc       integer,  
    primer_nombre varchar (20),  
    primer_apel   varchar (10),  
    segundo_apel  varchar(10),  
    fecha_Nac     date,  
    calle         varchar(10),  
    nro_calle     integer
```

Clave Primaria (PK) compuesta
Se define **sólo a nivel de tabla**

!!!

```
CONSTRAINT pk_tipo_nro_doc PRIMARY KEY (tipo_Doc, nro_doc));
```

Nivel de tabla

Empleado	
PK	<u>Tipo_Doc</u>
PK	<u>Nro_doc</u>
	Primer_Nombre
	Primer_Apel
	Segundo_Apel
	Fecha_Nac
	Calle
	Nro_calle

Crear una tabla: clave primaria

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoP_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio,  
    CONSTRAINT <nombre_restricción> PRIMARY KEY (atributo1_nombre,  
    atributo2_nombre,..., atributoP_nombre)  
);
```

```
CREATE TABLE empleado (  
    tipo_Doc            varchar(3),  
    nro_doc              integer,  
    primer_nombre       varchar (20),  
    primer_apel         varchar (10),  
    segundo_apel        varchar(10),  
    fecha_Nac           date,  
    calle               varchar(10),  
    nro_calle           integer  
    CONSTRAINT pk_tipo_nro_doc PRIMARY KEY (tipo_Doc, nro_doc) );
```

¿Cuántas PK se pueden definir por tabla?
¿La PK pueden contener valores NULL?

Empleado	
PK	<u>Tipo_Doc</u>
PK	<u>Nro_doc</u>
	Primer_Nombre
	Primer_Apel
	Segundo_Apel
	Fecha_Nac
	Calle
	Nro_calle

Crear una tabla: clave foránea (simple)

```
CREATE TABLE <nombre_tabla> (
    atributo1_nombre    tipo_dominio CONSTRAINT <nombre_restricción> REFERENCES <nombre_tabla_padre>
(<nombre_atributo_tabla_padre> [ON DELETE <acción>] [ON UPDATE <acción>] ),
    atributo2_nombre    tipo_dominio,
    .....,
    atributoN_nombre    tipo_dominio
);
```

(a)

Empleado	
PK	<u>DNI</u>
	Fecha_Nac
	Edad
	Primer_Nombre
	Primer_Apel
	Segundo_Apel
FK	Id_Dpto

Clave Foránea (FK) simple
Se define a nivel de columna (a)
o a nivel de tabla (b)

```
CREATE TABLE <nombre_tabla> (
    atributo1_nombre    tipo_dominio,
    atributo2_nombre    tipo_dominio,
    .....,
    atributoN_nombre    tipo_dominio,
    CONSTRAINT <nombre_restricción> FOREIGN KEY (atributo1_nombre) REFERENCES <nombre_tabla_padre>
(<nombre_atributo_tabla_padre> [ON DELETE <acción>] [ON UPDATE <acción>] )
);
```

(b)

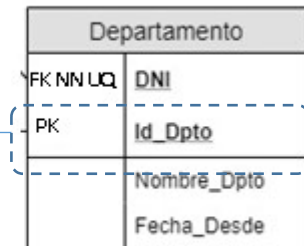
Crear una tabla: clave foránea (simple)

```
CREATE TABLE empleado (  
    DNI                integer PRIMARY KEY,  
    fecha_Nac          date,  
    edad               integer ,  
    primer_nombre      varchar (20),  
    primer_apel        varchar (10),  
    segundo_apel       varchar(10),  
    id_Dpto            integer CONSTRAINT fk_id_dpto REFERENCES departamento (id_dpto)  
);
```

(a)

```
CREATE TABLE empleado (  
    DNI                integer PRIMARY KEY,  
    fecha_Nac          date,  
    edad               integer ,  
    primer_nombre      varchar (20),  
    primer_apel        varchar (10),  
    segundo_apel       varchar(10),  
    id_Dpto            integer,  
    CONSTRAINT fk_id_dpto FOREIGN KEY (id_Dpto) REFERENCES departamento (id_dpto)  
);
```

(b)



← Nivel de columna

Clave Foránea(FK) simple
Se define a nivel de columna (a)
o a nivel de tabla (b)

← Nivel de tabla

Crear una tabla: clave foránea (simple)

```
CREATE TABLE empleado (  
    DNI                integer PRIMARY KEY,  
    fecha_Nac         date,  
    edad              integer ,  
    primer_nombre     varchar (20),  
    primer_apel       varchar (10),  
    segundo_apel      varchar(10),  
    id_Dpto           integer CONSTRAINT fk_id_dpto REFERENCES departamento (id_dpto)  
);
```

(a)

```
CREATE TABLE empleado (  
    DNI                integer PRIMARY KEY,  
    fecha_Nac         date,  
    edad              integer ,  
    primer_nombre     varchar (20),  
    primer_apel       varchar (10),  
    segundo_apel      varchar(10),  
    id_Dpto           integer,  
    CONSTRAINT fk_id_dpto FOREIGN KEY (id_Dpto) REFERENCES departamento (id_dpto)  
);
```

(b)

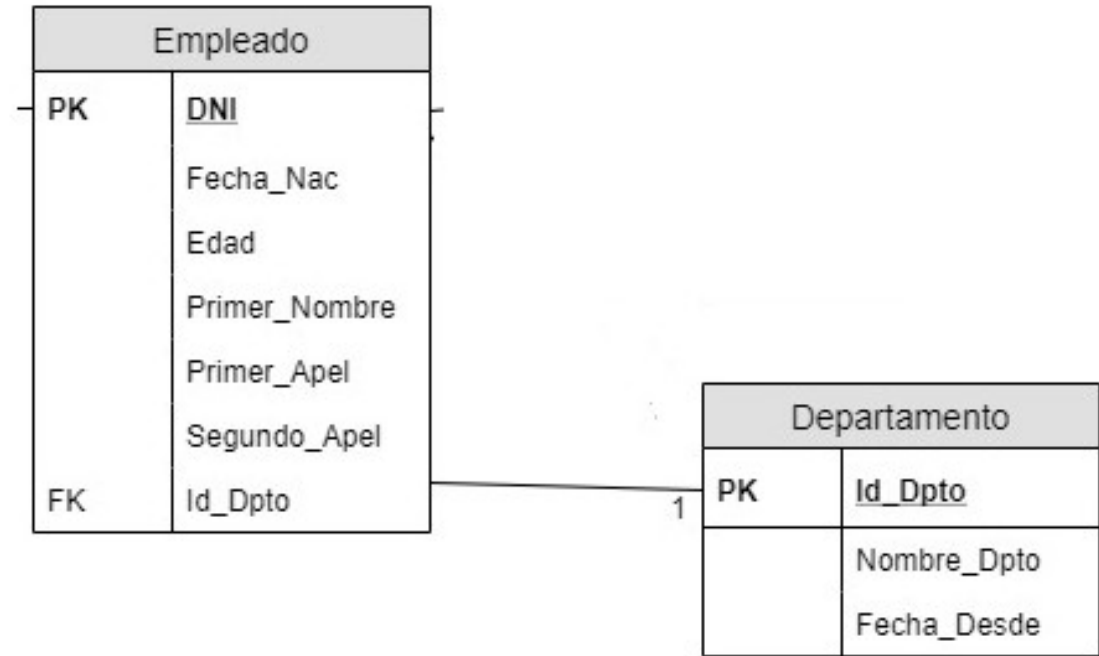
Empleado	
PK	<u>DNI</u>
	Fecha_Nac
	Edad
	Primer_Nombre
	Primer_Apel
	Segundo_Apel
FK	Id_Dpto

**Si sólo tengo creada
la tabla empleado
¿será correcta la
ejecución
de estos comandos?**

Crear una tabla: clave foránea

```
CREATE TABLE departamento(  
    Id_Dpto      integer PRIMARY KEY,  
    nombre_Dpto  varchar (20),  
    fecha_Desde  date  
);
```

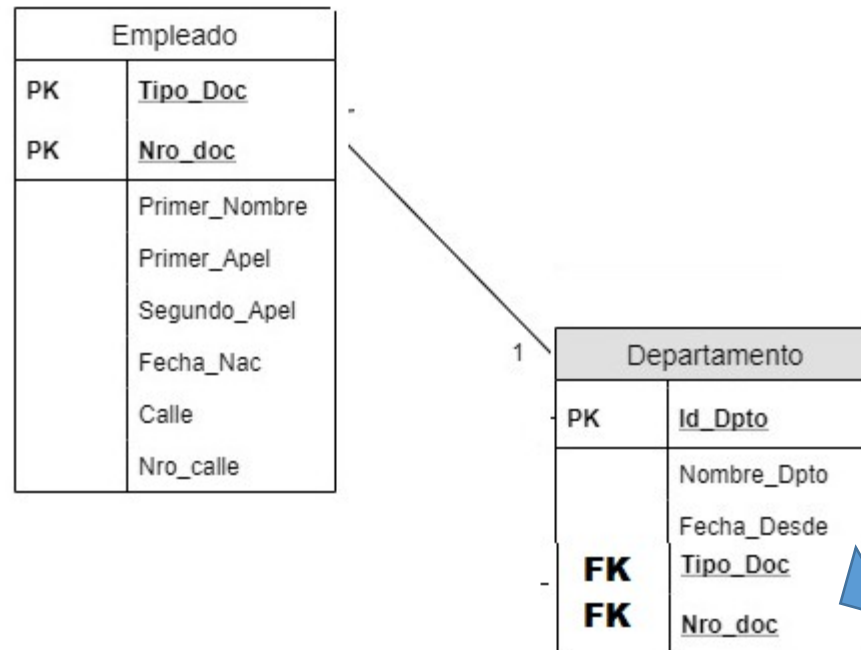
```
CREATE TABLE empleado (  
    DNI          integer PRIMARY KEY,  
    fecha_Nac    date,  
    edad         integer ,  
    primer_nombre varchar (20),  
    primer_apel  varchar (10),  
    segundo_apel varchar(10),  
    id_Dpto      integer CONSTRAINT fk_id_dpto REFERENCES departamento (id_dpto)  
);
```



**La tabla padre debe estar creada antes
que se cree la integridad referencial en la
tabla hija**

Crear una tabla: clave foránea(compuesta)

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoP_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio,  
    CONSTRAINT <nombre_restricción> FOREIGN KEY (atributo1_nombre,..., atributoP_nombre) REFERENCES  
    <nombre_tabla_padre> (<atributo1_nombre_tabla_padre,..., atributoP_nombre_tabla_padre>) [ON DELETE <acción>] [ON  
    UPDATE <acción>]  
);
```



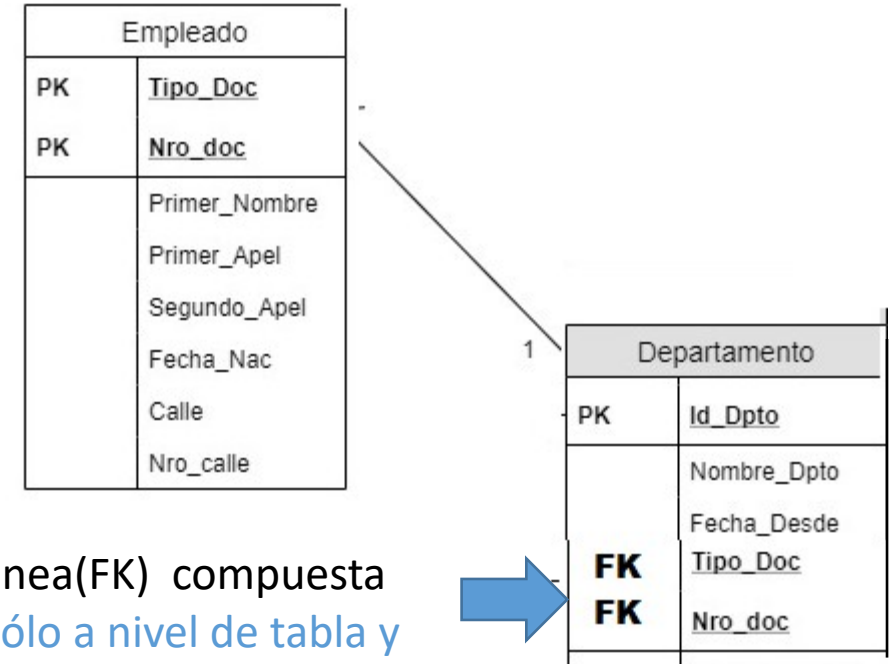
Clave Foránea(FK) compuesta
Se define sólo a nivel de tabla y con
la tabla padre creada previamente

!!!

Crear una tabla: clave foránea (compuesta)

```
CREATE TABLE empleado (  
    tipo_doc          varchar (3),  
    Nro_doc           integer ,  
    fecha_Nac         date,  
    primer_nombre     varchar (20),  
    primer_apel       varchar (10),  
    segundo_apel      varchar(10),  
    calle             varchar(10),  
    nro_calle         integer,  
    CONSTRAINT pk_doc PRIMARY KEY (tipo_doc,nro_doc)  
);
```

```
CREATE TABLE departamento(  
    id_dpto           integer pk_id_dpto PRIMARY KEY,  
    nombre_dpto       varchar (20),  
    fecha_desde       date,  
    tipo_doc          varchar (3),  
    nro_doc           integer,  
    CONSTRAINT fk_doc FOREIGN KEY (tipo_doc,nro_doc) REFERENCES empleado (tipo_doc,nro_doc)  
);
```



Clave Foránea(FK) compuesta
Se define sólo a nivel de tabla y
si la tabla padre ha sido creada
previamente.

!!!

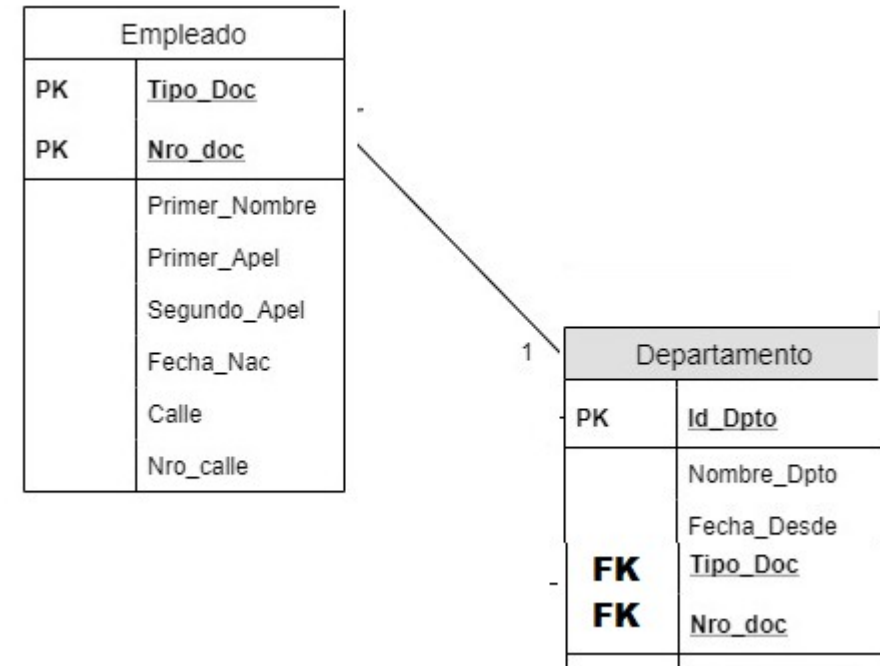
← Nivel de tabla

Crear una tabla: clave foránea

```
CREATE TABLE empleado (  
    tipo_doc          varchar (3),  
    Nro_doc           integer ,  
    fecha_Nac         date,  
    primer_nombre     varchar (20),  
    primer_apel       varchar (10),  
    segundo_apel      varchar(10),  
    calle             varchar(10),  
    nro_calle         integer,  
    CONSTRAINT pk_doc PRIMARY KEY (tipo_doc,nro_doc)  
);
```

```
CREATE TABLE departamento(  
    id_dpto           integer pk_id_dpto PRIMARY KEY,  
    nombre_dpto       varchar (20),  
    fecha_desde       date,  
    tipo_doc          varchar (3),  
    nro_doc           integer,  
    CONSTRAINT fk_doc FOREIGN KEY (tipo_doc,nro_doc) REFERENCES empleado (tipo_doc,nro_doc)  
);
```

¿Cuántas FK se pueden definir por tabla?



Crear una tabla: claves foráneas- acciones

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoP_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio,  
    CONSTRAINT <nombre_restricción> FOREIGN KEY (atributo1_nombre,..., atributoP_nombre) REFERENCES  
    <nombre_tabla_padre> (<atributo1_nombre_tabla_padre,..., atributoP_nombre_tabla_padre>)  
    [ON DELETE <acción>] [ON UPDATE <acción>]  
);
```

Restrict

Set Null

Cascade

Set Default

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ RESTRICT

Es el comportamiento por defecto, que impide realizar modificaciones/borrados que atentan contra la integridad referencial.

```
CREATE TABLE ciudades (ciudad_id integer PRIMARY KEY,  
                        ciudad_nom varchar(70));
```

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                        Persona_nom varchar(70),  
                        Ciudad_id CONSTRAINT fk_ciudad REFERENCES  
ciudades(ciudad_id));
```

ó

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                        Persona_nom varchar(70),  
                        Ciudad_id CONSTRAINT fk_ciudad REFERENCES  
ciudades(ciudad_id) ON DELETE RESTRICT ON UPDATE RESTRICT;
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Las dos definiciones de la clave foránea en la sentencia CREATE TABLE tienen el mismo efecto

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ RESTRICT

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                        Persona_nom varchar(70),  
                        Ciudad_id CONSTRAINT fk_ciudad  
REFERENCES ciudades(ciudad_id) ON DELETE RESTRICT);
```

```
DELETE FROM ciudades WHERE ciudad_id = 4;
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

Tabla Hija

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Tabla Padre

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ RESTRICT

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                        Persona_nom varchar(70),  
                        Ciudad_id CONSTRAINT fk_ciudad  
REFERENCES ciudades(ciudad_id) ON DELETE RESTRICT);
```

```
DELETE FROM ciudades WHERE ciudad_id = 4;
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

Tabla Hija

RESTRICT

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalén

Tabla Padre

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ RESTRICT

```
CREATE TABLE persona (persona_id integer  
PRIMARY KEY,
```

Persona_nom

```
varchar(70),
```

Ciudad_id CONSTRAINT

```
fk_ciudad REFERENCES ciudades(ciudad_id) ON  
DELETE RESTRICT
```

```
);
```

```
DELETE FROM ciudades WHERE ciudad_id = 4;
```

```
DELETE FROM ciudades WHERE ciudad_id = 1;
```



Mensaje de Error

Cannot delete or update a parent row: a foreign key constraint fails

```
(db.personas, CONSTRAINT personas_ibfk_1 FOREIGN KEY (ciudad_id) REFERENCES ciudades (ciudad_id))
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

* Si intentamos borrar una fila en **ciudades** que tenga valores en **personas** tendremos el error
foreign key constraint fails

* Si no hay registro relacionado se borrará

RESTRICT

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalén

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ CASCADE

Borra las tuplas de la tabla hija cuando se borra la tupla a la que hace referencia en la tabla padre, o actualiza el valor de la clave foránea cuando se actualiza el valor en la tupla referenciada en la tabla padre.

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                        Persona_nom varchar(70),  
                        Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id) ON DELETE CASCADE;  
);
```

```
DELETE FROM ciudades WHERE ciudad_id = 4;
```

```
DELETE FROM ciudades WHERE ciudad_id = 1;
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

Tabla Hija

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalén

Tabla Padre

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ CASCADE

```
CREATE TABLE persona (  
  persona_id integer PRIMARY KEY,  
  Persona_nom varchar(70),  
  Ciudad_id CONSTRAINT fk_ciudad REFERENCES  
  ciudades(ciudad_id) ON DELETE CASCADE  
);
```

```
DELETE FROM ciudades WHERE ciudad_id = 1;
```

personas		
persona_id	persona_nom	ciudad_id
1	David	1
2	Santiago	2
3	Juan	3
4	Antonio	1

* Si intentamos borrar una fila en **ciudades** que tenga valores en **personas** todos los registros relacionados en **personas** se borrarán

CASCADE

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalén

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ CASCADE

```
CREATE TABLE personas (persona_id integer PRIMARY KEY,  
                        Persona_nom varchar(70),  
                        Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id) ON DELETE CASCADE);
```

```
CREATE TABLE autos (motor integer PRIMARY KEY,  
                    modelo integer,  
                    id_pers CONSTRAINT fk_auto REFERENCES personas(personas_id) ON DELETE CASCADE);
```

autos		
Motor	Modelo	id_pers
1125	2008	1
2358	2015	2
3258	2020	1
5879	2018	3

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalén

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

☐ CASCADE

autos		
Motor	Modelo	id_pers
1125	2008	1
2358	2015	2
3258	2020	1
5879	2018	3

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

DELETE FROM ciudad WHERE ciudad_id = 1;

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ CASCADE

```
CREATE TABLE personas (persona_id integer PRIMARY KEY,  
                        Persona_nom varchar(70),  
                        Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id) ON DELETE CASCADE);
```

```
CREATE TABLE autos (motor integer PRIMARY KEY,  
                    modelo integer,  
                    id_pers CONSTRAINT fk_auto REFERENCES personas(personas_id) ;
```

autos		
Motor	Modelo	id_pers
1125	2008	1
2358	2015	2
3258	2020	1
5879	2018	3

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalén

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

☐ CASCADE

autos		
Motor	Modelo	id_pers
1125	2008	1
2358	2015	2
3258	2020	1
5879	2018	3

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

ERROR: update o delete en «personas» viola la llave foránea «autos_persona_id_fkey» en la tabla «autos»
DETAIL: La llave (persona_id)=(1) todavía es referida desde la tabla «autos».

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

❑ SET NULL/ SET DEFAULT

Establece NULL o el valor por DEFAULT a las claves foráneas en la tabla hija cuando se elimina la tupla en la tabla padre o se modifica el valor de la columna referenciada de la tabla padre.

```
CREATE TABLE persona (persona_id integer PRIMARY KEY, Persona_nom varchar(70),  
Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id) ON DELETE SET NULL);
```

```
DELETE FROM ciudades WHERE ciudad_id = 1;
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	NULL
2	Santiago	2
3	Juan	3
4	Andrés	NULL

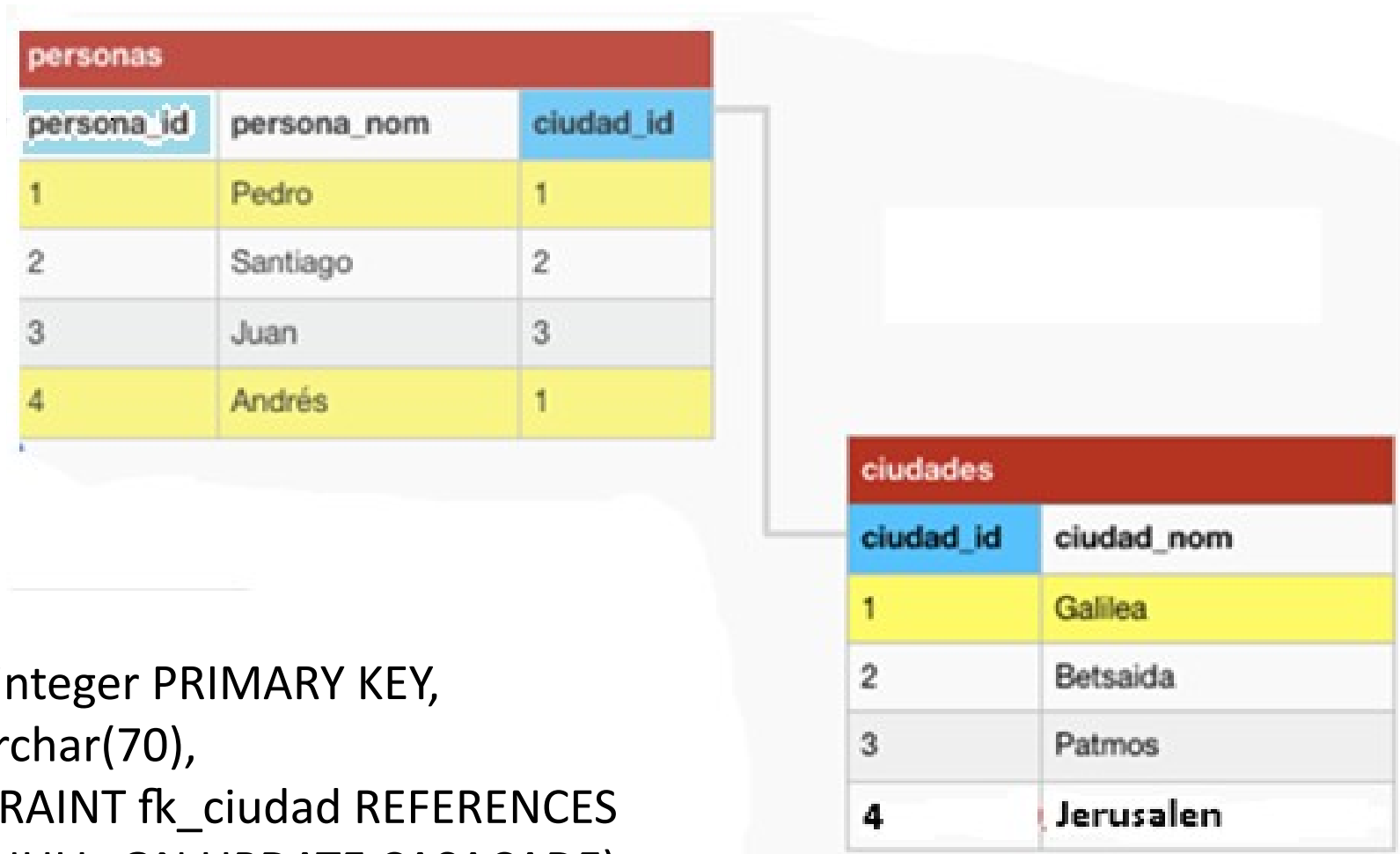
* Si intentamos borrar una fila en **ciudades** que tenga valores en **personas** todos los registros relacionados en **personas** tendrán el valor **NULL** en la columna que es llave foránea

SET NULL

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalén

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea



```
CREATE TABLE personas (persona_id integer PRIMARY KEY,  
                          Persona_nom varchar(70),  
                          Ciudad_id CONSTRAINT fk_ciudad REFERENCES  
ciudades(ciudad_id) ON DELETE SET NULL ON UPDATE CASACADE);
```

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

```
CREATE TABLE personas (persona_id integer PRIMARY KEY,  
                          Persona_nom varchar(70),  
                          Ciudad_id CONSTRAINT fk_ciudad  
REFERENCES ciudades(ciudad_id) ON DELETE SET NULL ON  
UPDATE CASCADE);
```

```
DELETE FROM ciudad WHERE ciudad_id = 1;
```

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Crear una tabla: claves foráneas- acciones

Acciones asociadas con ON DELETE y ON UPDATE definidas en una clave foránea

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	25
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
25	Betsaida
3	Patmos
4	Jerusalen

```
CREATE TABLE personas (persona_id integer PRIMARY KEY,  
                          Persona_nom varchar(70),  
                          Ciudad_id CONSTRAINT fk_ciudad REFERENCES  
ciudades(ciudad_id) ON DELETE SET NULL ON UPDATE CASCADE);
```

```
UPDATE ciudades SET ciudad_id = 25 where ciudades.ciudad_id = 2
```

Crear tabla: restricción Not Null

Restricciones:

Clave Primaria ✓

Clave Foránea ✓

Not Null

Clave Única

Check

La restricción NOT NULL, sólo se define a nivel de columna en la cláusula CREATE TABLE....

```
CREATE TABLE empleado (  
    DNI                integer PRIMARY KEY,  
    fecha_Nac          date,  
    edad               integer DEFAULT (18) ,  
    primer_nombre       varchar (20) NOT NULL,  
    primer_apel         varchar (10) NOT NULL,  
    segundo_apel        varchar(10),  
    id_Dpto             integer CONSTRAINT fk_id_dpto REFERENCES  
departamento (id_dpto)
```

Empleado	
PK	<u>DNI</u>
	Fecha_Nac
	Edad
NN	Primer_Nombre
NN	Primer_Apel
	Segundo_Apel
FK	Id_Dpto

Crear tabla: valores predeterminados

La restricción DEFAULT, **sólo se define a nivel de columna** en la cláusula CREATE TABLE.... Se la utiliza cuando se quiere dar un valor automáticamente a la dada columna.

Restricciones:

Clave Primaria ✓

Clave Foránea ✓

Not Null ✓

Clave Única

Check

```
CREATE TABLE empleado (  
    DNI                integer PRIMARY KEY,  
    fecha_Nac          date,  
    edad               integer DEFAULT (18) ,  
    primer_nombre      varchar (20) NOT NULL,  
    primer_apel        varchar (10) NOT NULL,  
    segundo_apel       varchar(10),  
    id_Dpto            integer CONSTRAINT fk_id_dpto REFERENCES  
departamento (id_dpto)  
);
```

Empleado	
PK	<u>DNI</u>
	Fecha_Nac
	Edad
NN	Primer_Nombre
NN	Primer_Apel
	Segundo_Apel
FK	Id_Dpto

Crear tabla: restricción UNIQUE

- Estable que los valores de una columna o conjunto de columnas deben ser únicos para todas las filas de la tabla.
- Pueden existir más de una UK por tabla
- En restricciones compuestas que admiten valores nulos, dos filas tendrán igual valor de clave si tienen los mismos valores en los campos que no son nulos.

Restricciones:

Clave Primaria ✓✓

Clave Foránea ✓✓

Not Null ✓✓

Clave Única

Check

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio CONSTRAINT <nombre_restricción> UNIQUE KEY,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoP_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio);
```

Restricción UNIQUE simple se define a nivel de columna o a nivel de tabla.
Restricción UNIQUE compuesta sólo se puede definir a nivel de tabla.

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoP_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio,  
    CONSTRAINT <nombre_restricción> UNIQUE KEY (atributo1_nombre,..., atributoP_nombre));
```

Crear tabla: restricción UNIQUE

Restricciones:

Clave Primaria ✓

Clave Foránea ✓

Not Null ✓

Clave Única

Check

```
CREATE TABLE departamento(  
    id_dpto          integer pk_id_dpto PRIMARY KEY,  
    nombre_dpto      varchar (20) uk_nom_dpto UNIQUE KEY, ← Nivel de columna  
    fecha_desde      date,  
    tipo_doc         varchar (3),  
    nro_doc          integer,  
    CONSTRAINT fk_doc FOREIGN KEY (tipo_doc,nro_doc) REFERENCES empleado (tipo_doc,nro_doc),  
    CONSTRAINT Uk_doc UNIQUE KEY (tipo_doc, nro_doc) ← Nivel de tabla  
);
```

Se debe expresar explícitamente cada tipo de restricción que afecta a cada columna.

Departamento	
PK	<u>Id_Dpto</u>
UK	<u>Nombre_Dpto</u>
	Fecha_Desde
FK UK	<u>Tipo_Doc</u>
FK UK	<u>Nro_doc</u>

Crear tabla: restricción CHECK

Restricciones:

Clave Primaria ✓

Clave Foránea ✓

Not Null ✓

Clave Única ✓

Check

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio CONSTRAINT <nombre_restricción> CHECK <expresión>,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoP_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio,  
);
```

Restricción CHECK se puede definir a nivel de columna o a nivel de tabla.

```
CREATE TABLE <nombre_tabla> (  
    atributo1_nombre    tipo_dominio,  
    atributo2_nombre    tipo_dominio,  
    .....,  
    atributoP_nombre    tipo_dominio,  
    .....,  
    atributoN_nombre    tipo_dominio,  
    CONSTRAINT <nombre_restricción> CHECK <expresión>  
);
```

Crear tabla: restricción CHECK

Restricciones:

Clave Primaria ✓✓

Clave Foránea ✓✓

Not Null ✓✓

Clave Única ✓✓

Check

```
CREATE TABLE departamento(  
    id_dpto          integer pk_id_dpto PRIMARY KEY,  
    nombre_dpto      varchar (20) uk_nom_dpto UNIQUE KEY,  
    fecha_desde      date CHECK (fecha_desde <= today), ← Nivel de columna  
    tipo_doc          varchar (3),  
    nro_doc           integer,  
    CONSTRAINT fk_doc FOREIGN KEY (tipo_doc,nro_doc) REFERENCES empleado (tipo_doc,nro_doc),  
    CONSTRAINT uk_doc UNIQUE KEY (tipo_doc, nro_doc),  
    CONSTRAINT ch_tipo_doc CHECK (tipo_doc = 'DNI' or tipo_doc = 'LC' or tipo_doc = 'LE') ← Nivel de tabla  
);
```

Departamento	
PK	<u>Id_Dpto</u>
UK	Nombre_Dpto
Check	Fecha_Desde
FK UK	<u>Tipo_Doc</u>
FK UK	<u>Nro_doc</u>

- ✓ Exige la integridad de domino mediante la limitación de valores que puede aceptar una columna.
- ✓ Determina los valores válidos a partir de una expresión lógica.
- ✓ Especifica una condición que debe ser verdadera.

Crear tabla: restricción CHECK

Restricciones:

Clave Primaria ✓✓

Clave Foránea ✓✓

Not Null ✓✓

Clave Única ✓✓

Check

```
CREATE TABLE departamento(  
    id_dpto          integer pk_id_dpto PRIMARY KEY,  
    nombre_dpto      varchar (20) uk_nom_dpto UNIQUE KEY,  
    fecha_desde      date CHECK (fecha_desde <= today),  
    tipo_doc          varchar (3),  
    nro_doc           integer,  
    CONSTRAINT fk_doc FOREIGN KEY (tipo_doc,nro_doc) REFERENCES empleado (tipo_doc,nro_doc),  
    CONSTRAINT uk_doc UNIQUE KEY (tipo_doc, nro_doc),  
    CONSTRAINT ch_tipo_doc CHECK (tipo_doc = 'DNI' or tipo_doc = 'LC' or tipo_doc = 'LE')  
);
```

Nivel de columna

Nivel de tabla

Departamento	
PK	<u>Id_Dpto</u>
UK	Nombre_Dpto
Check	Fecha_Desde
FK UK	<u>Tipo_Doc</u>
FK UK	<u>Nro_doc</u>

Una columna puede estar afectada por más de una restricción
Se deben definir todas las restricciones necesarias para cada columna

DROP

Esquema

- Crear CREATE SCHEMA <nom_esquema>
- Borrar DROP SCHEMA <nom_esquema>
- Cambiar definición ALTER SCHEMA <nom_esquema>

Tablas

- Crear CREATE TABLE <nom_tabla> (<nom_col> <tipo_col> [restricciones_columna], [...] [restricciones_tabla]);
- Donde *restricciones_columna* es: [CONSTRAINT constraint_name] { NOT NULL | NULL | CHECK (expression) | DEFAULT default_expr | UNIQUE | PRIMARY KEY | REFERENCES <reftable> [(<refcolumn>)] [ON DELETE referential_action] [ON UPDATE referential_action] }
- Donde *restricciones_tabla* es: [CONSTRAINT <constraint_name>] { CHECK (expression) | UNIQUE (<column_name> [, ...]) | PRIMARY KEY (<column_name> [, ...]) | FOREIGN KEY (<column_name> [, ...]) REFERENCES <reftable> [(<refcolumn> [, ...])] [ON DELETE referential_action] [ON UPDATE referential_action] }
- Borrar DROP TABLE <nom_tabla>
- Cambiar nombre tabla ALTER TABLE <nom_tabla> RENAME TO <nuevo_nom_tabla>
- Cambiar

Columnas

- Añadir ALTER TABLE <nom_tabla> ADD COLUMN <nom_col> <tipo_col>;
- Borrar ALTER TABLE <nom_tabla> DROP COLUMN <nom_col>;
- Cambiar Nombre ALTER TABLE <nom_tabla> RENAME COLUMN <nom_col> TO <nuevo_nom_col>;
- Cambiar Tipo de dato ALTER TABLE <nom_table> ALTER COLUMN <nom_col> TYPE <tipo_col>;
- Cambiar Longitud ALTER TABLE <nom_col> ALTER COLUMN <nom_col> TYPE <tipo_col> <nueva_long>;

Restricciones

- Añadir ALTER TABLE <nom_tabla> ALTER COLUMN <nom_col> SET NOT NULL
- ALTER TABLE <nom_tabla> ADD CONSTRAINT { CHECK (expression) | UNIQUE (<column_name> [, ...]) | FOREIGN KEY (<column_name> [, ...]) REFERENCES <reftable> [(<refcolumn> [, ...])] [ON DELETE referential_action] [ON UPDATE referential_action] }
- Borrar ALTER TABLE <nom_tabla> ALTER COLUMN <nom_col> DROP NOT NULL
- ALTER TABLE <nom_tabla> DROP CONSTRAINT <nom_restricción> { CHECK | FOREIGN | UNIQUE }

Borrado de tabla

- La cláusula DROP hace que la tabla pierda sus datos e índices asociados.
- Es una acción irreversible.

```
DROP TABLE <nombre_tabla>;
```

```
DROP TABLE departamento;
```

Base de Datos

Objetos /Estructura DDL

Esquema

- Crear CREATE SCHEMA
- Borrar DROP SCHEMA
- Cambiar definición ALTER SCHEMA

Tablas

- Crear CREATE TABLE
- Borrar DROP TABLE..
- Cambiar nombre tabla ALTER TABLE.....RENAME TO
- Cambiar

Columnas

- Añadir ALTER TABLE ADD COLUMN.....
- Borrar ALTER TABLE DROP COLUMN.....
- Cambiar Nombre ALTER TABLE RENAME COLUMN TO.....
- Cambiar Tipo de dato ALTER TABLE ALTER COLUMN TYPE.....
- Cambiar Longitud ALTER TABLE ALTER COLUMN TYPE.....

Restricciones

- Añadir ALTER TABLE ALTER COLUMN..... SET NOT NULL
- ALTER TABLE..... ADD CONSTRAINT CHECK/FOREIGN/UNIQUE...
- Borrar ALTER TABLE ALTER COLUMN DROP NOT NULL
- ALTER TABLE DROP CONSTRAINTCHECK/FOREIGN/UNIQUE...

Filas

- Crear INSERT INTO VALUES (.....)
- Modificar UPDATE SET
- Borrar DELETE FROM

Datos DML

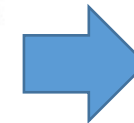
Modificar una tabla

- La cláusula ALTER permite introducir cambios en la estructura de una tabla de la base de datos.
- Renombrar una tabla de la base de datos.

```
ALTER TABLE <nombre_tabla> RENAME TO <nuevo_nombre>;
```

```
ALTER TABLE departamento RENAME TO dpto_empresa;
```

Departamento	
PK	<u>Id_Dpto</u>
UK	Nombre_Dpto
Check	Fecha_Desde
FK UK	<u>Tipo_Doc</u>
FK UK	<u>Nro_doc</u>



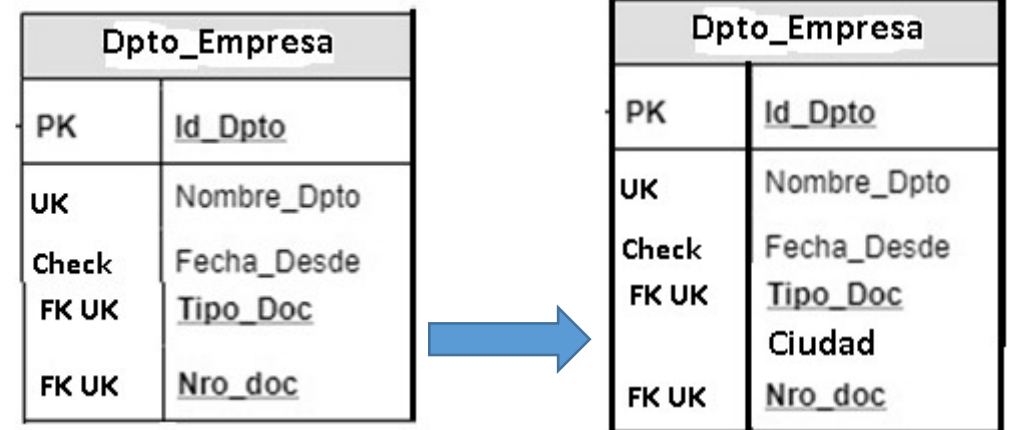
Dpto_Empresa	
PK	<u>Id_Dpto</u>
UK	Nombre_Dpto
Check	Fecha_Desde
FK UK	<u>Tipo_Doc</u>
FK UK	<u>Nro_doc</u>

Modificar una tabla: añadir columnas

- No se puede especificar la posición de la nueva columna en la tabla
- Si la tabla ya posee tuplas almacenadas, la nueva columna se carga con NULL
- Sólo en tablas vacías, las columnas añadidas se pueden crear con restricción NOT NULL

```
ALTER TABLE <nombre_tabla> ADD COLUMN <nombre_nueva_columna> <tipo_nueva_columna>;
```

```
ALTER TABLE dpto_empresa ADD COLUMN ciudad varchar(20);
```



Modificar una tabla: borrar columna

```
ALTER TABLE <nombre_tabla> DROP COLUMN <nombre_columna>;
```

```
ALTER TABLE dpto_empresa DROP COLUMN ciudad;
```

Dpto_Empresa	
PK	<u>Id_Dpto</u>
UK	Nombre_Dpto
Check	Fecha_Desde
FK UK	<u>Tipo_Doc</u>
	Ciudad
FK UK	<u>Nro_doc</u>

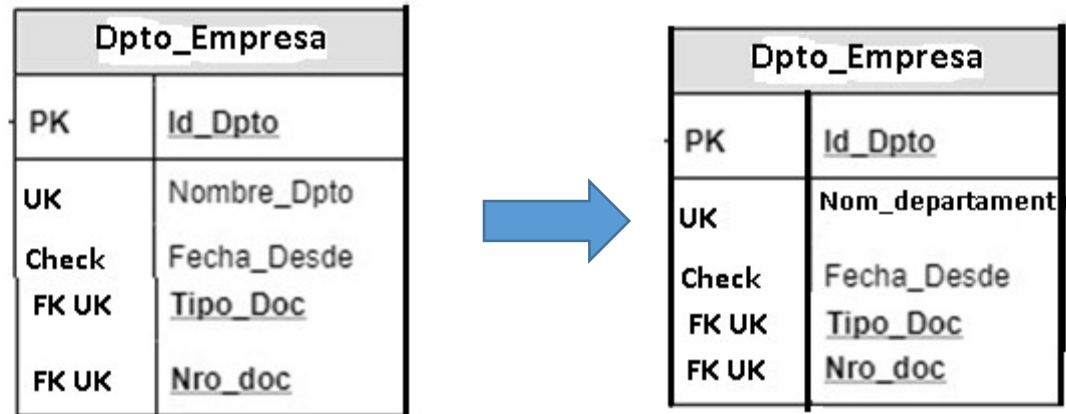


Dpto_Empresa	
PK	<u>Id_Dpto</u>
UK	Nombre_Dpto
Check	Fecha_Desde
FK UK	<u>Tipo_Doc</u>
FK UK	<u>Nro_doc</u>

Modificar una tabla: cambiar nombre columna

```
ALTER TABLE <nombre_tabla> RENAME COLUMN <nombre_columna> TO <nuevo_nombre_columna>;
```

```
ALTER TABLE dpto_empresa RENAME COLUMN nombre_dpto TO nom_departamento;
```



Modificar una tabla: cambiar tipo/longitud columna

```
ALTER TABLE <nombre_tabla> ALTER COLUMN <nombre_columna> TYPE <nuevo_tipo>;
```

```
ALTER TABLE <nombre_tabla> ALTER COLUMN <nombre_column> TYPE <tipo_col> <nueva long>
```

```
ALTER TABLE dpto_empresa ALTER COLUMN nom_departamento TYPE varchar (30) ;
```


Base de Datos

Objetos /Estructura DDL

Esquema

- Crear CREATE SCHEMA
- Borrar DROP SCHEMA
- Cambiar definición ALTER SCHEMA

Tablas

- Crear CREATE TABLE
- Borrar DROP TABLE..
- Cambiar nombre tabla ALTER TABLE.....
- Cambiar

Columnas

- Añadir ALTER TABLE ADD COLUMN.....
- Borrar ALTER TABLE DROP COLUMN.....
- Cambiar Nombre ALTER TABLE RENAME COLUMN TO.....
- Cambiar Tipo de dato ALTER TABLE ALTER COLUMN TYPE.....
- Cambiar Longitud ALTER TABLE ALTER COLUMN TYPE.....

Restricciones

- Añadir ALTER TABLE ALTER COLUMN..... SET NOT NULL
- ALTER TABLE..... ADD CONSTRAINT CHECK/FOREIGN/UNIQUE...
- Borrar ALTER TABLE ALTER COLUMN DROP NOT NULL
- ALTER TABLE DROP CONSTRAINTCHECK/FOREIGN/UNIQUE...

Datos DML

Filas

- Crear INSERT INTO VALUES (.....)
- Modificar UPDATE SET
- Borrar DELETE FROM

Modificar una tabla: añadir restricción

```
ALTER TABLE <nombre_tabla> ALTER COLUMN <nombre_columna> SET NOT NULL;
```

```
ALTER TABLE <nombre_tabla> ADD CONSTRAINT <nombre_restriccion> CHECK/FOREIGN/UNIQUE
```

```
ALTER TABLE dpto_empresa ALTER COLUMN nombre_dpto SET NOT NULL ;
```

```
ALTER TABLE dpto_empresa ADD CONSTRAINT nombre_dpto UNIQUE KEY;
```

Modificar una tabla: añadir restricción

```
ALTER TABLE <nombre_tabla> ALTER COLUMN <nombre_columna> SET NOT NULL;
```

```
ALTER TABLE <nombre_tabla> ADD CONSTRAINT <nombre_restriccion> CHECK/FOREIGN/UNIQUE
```

**Prestar especial atención
a esta diferencia de la restricción
NOT NULL con las otras**

```
ALTER TABLE dpto_empresa ALTER COLUMN nombre_dpto SET NOT NULL ;
```

```
ALTER TABLE dpto_empresa ADD CONSTRAINT nombre_dpto UNIQUE KEY;
```

Modificar una tabla: borrar restricción

```
ALTER TABLE <nombre_tabla> ALTER COLUMN <nombre_columna> DROP NOT NULL;
```

```
ALTER TABLE <nombre_tabla> DROP CONSTRAINT <nombre_restriccion> CHECK/FOREIGN/UNIQUE
```

**Prestar especial atención
a esta diferencia de la restricción
NO NULL con las otras**

```
ALTER TABLE dpto_empresa ALTER COLUMN nombre_dpto DROP NOT NULL ;
```

```
ALTER TABLE dpto_empresa DROP CONSTRAINT nombre_dpto UNIQUE;
```

Base de Datos

Objetos /Estructura DDL

Esquema

- Crear CREATE SCHEMA
- Borrar DROP SCHEMA
- Cambiar definición ALTER SCHEMA

Tablas

- Crear CREATE TABLE
- Borrar DROP TABLE..
- Cambiar nombre tabla ALTER TABLE.....
- Cambiar

Columnas

- Añadir ALTER TABLE ADD COLUMN.....
- Borrar ALTER TABLE DROP COLUMN.....
- Cambiar Nombre ALTER TABLE RENAME COLUMN TO.....
- Cambiar Tipo de dato ALTER TABLE ALTER COLUMN TYPE.....
- Cambiar Longitud ALTER TABLE ALTER COLUMN TYPE.....

Restricciones

- Añadir ALTER TABLE ALTER COLUMN..... SET NOT NULL
- ALTER TABLE..... ADD CONSTRAINT CHECK/FOREIGN/UNIQUE...
- Borrar ALTER TABLE ALTER COLUMN DROP NOT NULL
- ALTER TABLE DROP CONSTRAINTCHECK/FOREIGN/UNIQUE...

Datos DML

Filas

- Crear INSERT INTO VALUES (.....)
- Modificar UPDATE SET
- Borrar DELETE FROM

Añadir filas o tuplas a una tabla

- ✓ Introduce instancias en las tuplas
- ✓ Si se omite el nombre de una columna, se insertará NULL o valor predeterminado en esa columna
- ✓ Si no se especifica las columnas se asume que en la cláusula VALUES vendrán los valores para todas las columnas en el orden en que fueron definidas
- ✓ Se considera una buena práctica especificar las columnas

```
INSERT INTO <nombre_tabla> (<nombre_columna1>,<nombre_columna2>....., <nombre_colunaN>) VALUES  
(<valor_columna1>,<valor_columna2>....., <nombre_colunaN>)
```

```
INSERT INTO departamento (id_dpto, nombre_dpt, ciudad) VALUES (1,'compras', 'Paraná') ;  
INSERT INTO departamento VALUES ( 1,'compras', 'Paraná');  
INSERT INTO departamento VALUES (1,'compras', ,);
```

Alternativa para
ingreso individual de
tuplas

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	NULL

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná

Añadir filas o tuplas a una tabla

```
INSERT INTO <nombre_tabla> (< nombre_columna1>,<nombre_columna2>,,,,,, <nombre_colunaN>) VALUES  
(<valor_columna1>,<valor_columna2>,,,,,, <nombre_colunaN>)
```

```
CREATE TABLE departamento(  
    id_dpto          integer pk_id_dpto PRIMARY KEY,  
    nombre_dpto      varchar (20) uk_nom_dpto UNIQUE KEY,  
    ciudad           varchar (20));
```

```
INSERT INTO departamento (id_dpto., nombre_dpt, ciudad) VALUES (1,'compras', '' );
```

```
INSERT INTO departamento (id_dpto, nombre_dpt, ciudad) VALUES ('compras', 1, 'Paraná' );
```

```
INSERT INTO departamento VALUES ( 'compras', 1 ,'Paraná');
```

```
INSERT INTO departamento VALUES ( , 'compras', ,);
```

```
INSERT INTO departamento VALUES (2, 'compras', Rosario);
```

¿Cuáles de las
sentencias INSERT son
correctas y cuales no?

Añadir filas o tuplas a una tabla

- ✓ Introduce instancias en las tuplas
- ✓ Si se omite el nombre de una columna, se insertará NULL o valor predeterminado en esa columna
- ✓ Si no se especifica las columnas se asume que en la cláusula VALUES vendrán los valores para todas las columnas en el orden en que fueron definidas, por ello debe dejar los espacios correspondiente.
- ✓ Se considera una buena práctica especificar las columnas

```
INSERT INTO <nombre_tabla> (<nombre_columna1>,<nombre_columna2>....., <nombre_colunaN>) VALUES  
(<valor_columna1>,<valor_columna2>....., <nombre_colunaN>)
```

```
INSERT INTO departamento VALUES (  
    (1,'compras', 'Paraná'), (2,'ventas', 'Paraná'), (3,'personal', "")  
);
```

```
INSERT INTO departamento (id_dpto., nombre_dpto., ciudad) VALUES (  
    (1,'compras', 'Paraná'), (2,'ventas', 'Paraná'), (3,'personal', "")  
);
```



Alternativas para ingreso de múltiples tuplas

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	NULL

Modificar filas o tuplas a una tabla

- ✓ Permite modificar valores en las tuplas de la base de datos
- ✓ Todas las tuplas que cumplan con la condición se modifican con los valores declarados en SET
- ✓ La condición refiere a una columna de la tabla, y puede contener :
 - comparaciones simples: =, >, <, >=, <=
 - comparaciones de rango: BETWEEN
 - expresiones lógicas: AND , OR, NOT
 - comparación con NULL : IS NULL, IS NOT NULL
 - otros: LIKE

```
UPDATE <nombre_tabla> SET < nombre_columna1> = <valor_columna1> WHERE <condición>;
```

```
UPDATE <nombre_tabla> SET < nombre_columna1> = <valor_columna1>;
```

```
UPDATE departamento SET ciudad = 'Rosario' WHERE id_dpto = 3;
```

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	NULL

Modificar filas o tuplas a una tabla

```
UPDATE <nombre_tabla> SET <nombre_columna1> = <valor_columna1> WHERE <condición>;
```

```
UPDATE <nombre_tabla> SET <nombre_columna1> = <valor_columna1>;
```

```
UPDATE departamento SET ciudad = 'Rosario' WHERE id_dpto = 3;
```

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	NULL



Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	Rosario



Modificar filas o tuplas a una tabla

```
UPDATE <nombre_tabla> SET < nombre_columna1> = <valor_columna1> WHERE <condición>;
```

```
UPDATE <nombre_tabla> SET < nombre_columna1> = <valor_columna1>;
```

```
UPDATE departamento SET ciudad = 'Rosario' ;
```

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	NULL

Modificar filas o tuplas a una tabla

```
UPDATE <nombre_tabla> SET <nombre_columna1> = <valor_columna1> WHERE <condición>;
```

```
UPDATE <nombre_tabla> SET <nombre_columna1> = <valor_columna1>;
```

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	NULL



```
UPDATE departamento SET ciudad = 'Rosario' ;
```



Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Rosario
2	ventas	Rosario
3	personal	Rosario

Borrar filas o tuplas a una tabla

- ✓ Permite eliminar tuplas de una tabla existente en la base de datos
- ✓ Todas las tuplas que cumplan con la condición del WHERE serán borradas
- ✓ La condición refiere a una columna de la tabla, y puede contener :
 - comparaciones simples: =, >, <, >=, <=
 - comparaciones de rango: BETWEEN
 - expresiones lógicas: AND , OR, NOT
 - comparación con NULL : IS NULL, IS NOT NULL
 - otros: LIKE

```
DELETE FROM <nombre_tabla> WHERE <condición>;
```

```
DELETE FROM departamento WHERE ciudad = 'Rosario' ;
```

Borrar filas o tuplas a una tabla

```
DELETE FROM <nombre_tabla> WHERE <condición>
```

```
DELETE FROM departamento WHERE ciudad = 'Rosario' ;
```

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	Rosario



Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná



> <

Borrar filas o tuplas a una tabla

```
DELETE FROM <nombre_tabla> WHERE <condición>
```

```
DELETE FROM departamento ;
```

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	Rosario

¿Qué ocurre en esta declaración sin where?

Borrar filas o tuplas a una tabla

```
DELETE FROM <nombre_tabla> WHERE <condición>
```

Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	Rosario



DELETE FROM departamento ;

Departamento		
Id_dpto	Nombre_dpto	ciudad

**Sólo queda la estructura de la tabla,
qué diferencia existe con el Drop Table?**

Borrar filas o tuplas a una tabla

```
DELETE FROM <nombre_tabla> WHERE <condición>
```

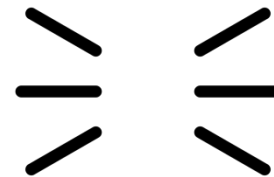
Departamento		
Id_dpto	Nombre_dpto	ciudad
1	compras	Paraná
2	ventas	Paraná
3	personal	Rosario

```
DELETE FROM departamento ;
```



Departamento		
Id_dpto	Nombre_dpto	ciudad

```
DROP TABLE departamento;
```



Sentencias DML y Restricción FOREIGN KEY

Sentencias DML permitidas por las acciones de integridad referencial cuando se ejecutan sobre claves primarias/únicas de las tabla padre, o sobre clave foránea de la tabla hijo

```
CREATE TABLE ciudades (ciudad_id integer PRIMARY KEY,  
                        ciudad_nom varchar(70));
```

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                       Persona_nom varchar(70),  
                       Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id));
```

```
Insert into ciudades values (5,'Catamarca');
```

```
Insert into personas values (5, 'Maria', 4);
```

```
Inset into personas values (5, 'Maria',10);
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Sentencias DML y Restricción FOREIGN KEY

Sentencias DML permitidas por las acciones de integridad referencial cuando se ejecutan sobre claves primarias/únicas de la tabla padre, o sobre clave foránea de la tabla hijo

```
CREATE TABLE ciudades (ciudad_id integer PRIMARY KEY,  
                        ciudad_nom varchar(70));
```

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                       Persona_nom varchar(70),  
                       Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id));
```

Update ciudades set ciudad_id = 10 where ciudad_id = 1;

Update ciudades set ciudad_id = 40 where ciudad_id = 4;

Update personas set ciudad_id = 20 where persona_id = 2;

Update personas set ciudad_id = 2 where persona_id = 3;

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Sentencias DML y Restricción FOREIGN KEY

Sentencias DML permitidas por las acciones de integridad referencial cuando se ejecutan sobre claves primarias/únicas de las tabla padre, o sobre clave foránea de la tabla hijo

```
CREATE TABLE ciudades (ciudad_id integer PRIMARY KEY,  
                        ciudad_nom varchar(70));
```

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                       Persona_nom varchar(70),  
                       Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id));
```

delete ciudades where ciudad_id = 1;

delete ciudades where ciudad_id = 4;

delete personas where persona_id = 2;

Update personas where persona_id = 1;

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Sentencias DML y Restricción FOREIGN KEY

Sentencias DML permitidas por las acciones de integridad referencial cuando se ejecutan sobre claves primarias/únicas de las tabla padre, o sobre clave foránea de la tabla hijo

```
CREATE TABLE ciudades (ciudad_id integer PRIMARY KEY,  
                        ciudad_nom varchar(70));
```

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                       Persona_nom varchar(70),  
                       Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id) ON DELETE cascade) ON UPDATE  
cascade ;
```

```
Insert into ciudades values (5,'Catamarca');
```

```
Insert into personas values (5, 'Maria', 4);
```

```
Inset into personas values (5, 'Maria',10);
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Sentencias DML y Restricción FOREIGN KEY

Sentencias DML permitidas por las acciones de integridad referencial cuando se ejecutan sobre claves primarias/únicas de la tabla padre, o sobre clave foránea de la tabla hijo

```
CREATE TABLE ciudades (ciudad_id integer PRIMARY KEY,  
                        ciudad_nom varchar(70));
```

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                       Persona_nom varchar(70),  
                       Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id) ON DELETE cascade) ON UPDATE  
cascade  
);
```

Update ciudades set ciudad_id = 10 where ciudad_id = 1;

Update ciudades set ciudad_id = 40 where ciudad_id = 4;

Update personas set ciudad_id = 20 where persona_id = 2;

Update personas set ciudad_id = 2 where persona_id = 3;

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Sentencias DML y Restricción FOREIGN KEY

Sentencias DML permitidas por las acciones de integridad referencial cuando se ejecutan sobre claves primarias/únicas de la tabla padre, o sobre clave foránea de la tabla hijo

```
CREATE TABLE ciudades (ciudad_id integer PRIMARY KEY,  
                        ciudad_nom varchar(70));
```

```
CREATE TABLE persona (persona_id integer PRIMARY KEY,  
                       Persona_nom varchar(70),  
                       Ciudad_id CONSTRAINT fk_ciudad REFERENCES ciudades(ciudad_id) ON DELETE cascade) ON UPDATE  
cascade  
);
```

```
delete ciudades where ciudad_id = 1;
```

```
delete ciudades where ciudad_id = 4;
```

```
delete personas where persona_id = 2;
```

```
Update personas where persona_id = 1;
```

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Sentencias DML y Restricción FOREIGN KEY

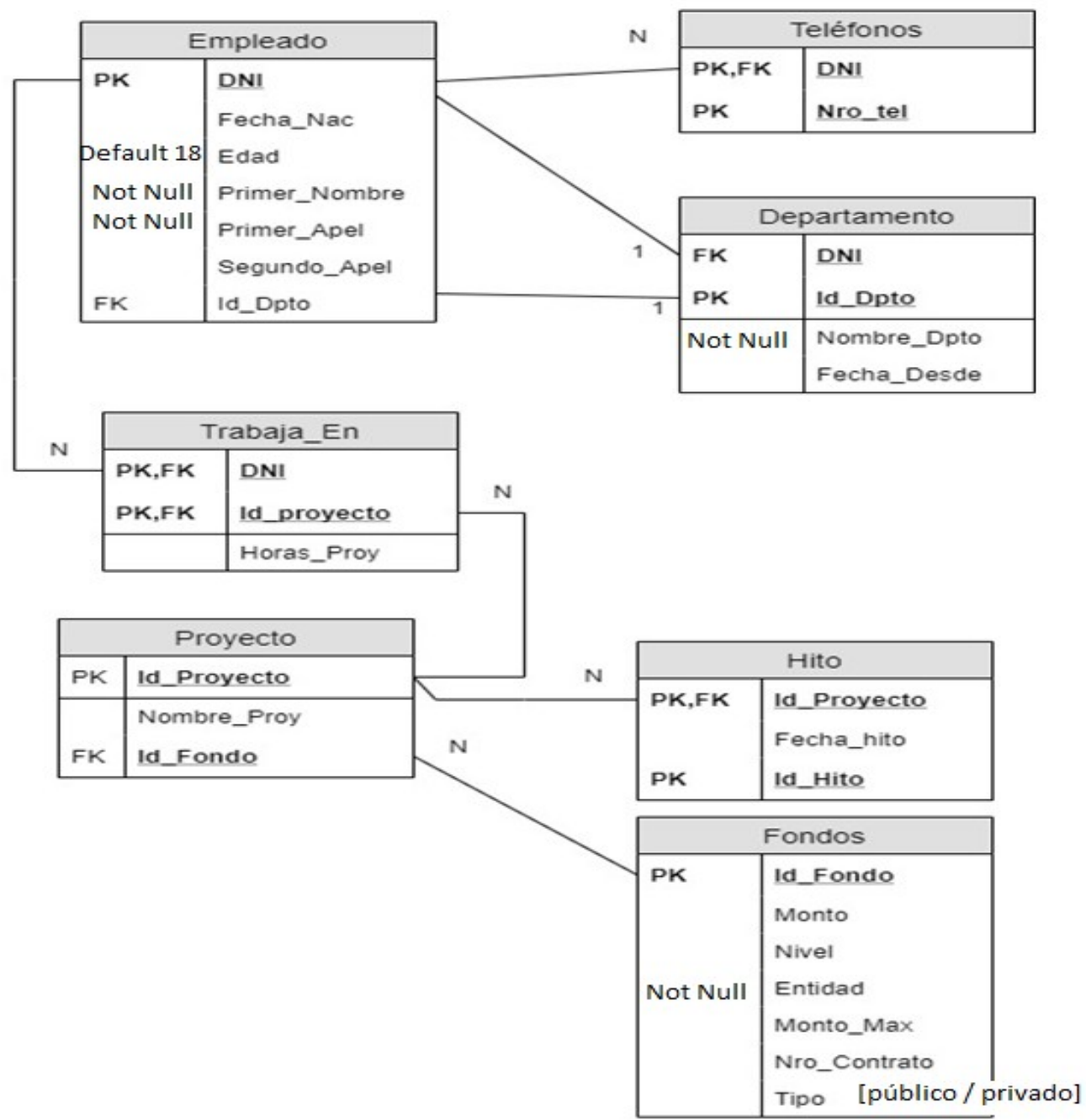
Sentencias DML permitidas por las acciones de integridad referencial cuando se ejecutan sobre claves primarias/únicas de las tabla padre, o sobre clave foránea de la tabla hijo

Sentencia DML	Sobre tabla PADRE	Sobre tabla HIJO
INSERT	Sin problemas, los valores de la clave son únicos	Sin problema si el valor de la clave foránea existe en la tabla padre o es parcial o totalmente nula
UPDATE	Permite si la sentencia no deja ninguna fila en la tabla hijo sin su correspondiente valor en la tabla padre	Permitida si la nueva clave foránea tiene un valor correspondiente con la tabla padre
DELETE restrict	Permita si ninguna fila de la tabla hijo hace referencia al valor de la clave en la tabla padre	Sin problema
DELETE cascade	Sin problema	Sin problema

personas		
persona_id	persona_nom	ciudad_id
1	Pedro	1
2	Santiago	2
3	Juan	3
4	Andrés	1

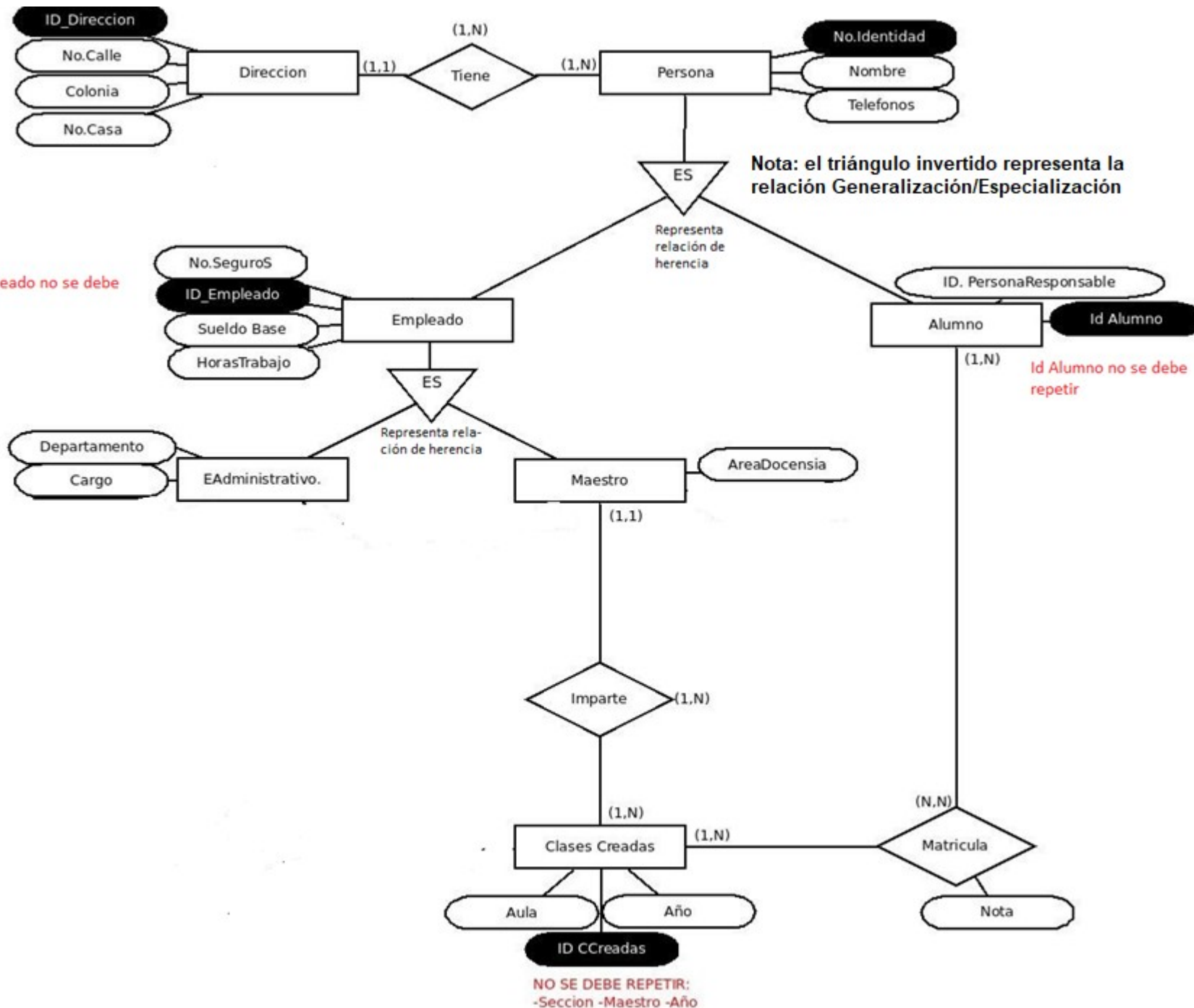
ciudades	
ciudad_id	ciudad_nom
1	Galilea
2	Betsaida
3	Patmos
4	Jerusalen

Insert into ciudades values (5,'Catamarca');
Insert into personas values (5, 'Maria', 4);
Inset into personas values (5, 'Maria',10);



Tarea 1: escribir las sentencias necesarias para crear las tablas asociadas con este diagrama de tablas. Los dominios de datos, para cada atributo, definirlos a propia elección.

Id_Empleado no se debe repetir



Tarea 2:

2.1 Transformar el siguiente diagrama DER en el correspondiente DT.

2.2. Escribir las sentencias necesarias para crear las tablas asociadas. Los dominios de datos, para cada atributo, definirlos a propia elección.